ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/327389927

Big Data Ecosystem: Review on Architectural Evolution: Proceedings of IEMIS 2018, Volume 2

Chapter · January 2019

DOI: 10.1007/978-981-13-1498-8_30

CITATION
1

READS
541

3 authors:



Kamakhya Singh
KIIT University
10 PUBLICATIONS **6** CITATIONS
SEE PROFILE



Rajat Kumar Behera
KIIT, Deemed to be University
17 PUBLICATIONS **25** CITATIONS
SEE PROFILE



Jibendu KUMAR Mantri
North Orissa University
29 PUBLICATIONS **52** CITATIONS
SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Blockchain View project

A Text Book of Advanced Java View project

All content following this page was uploaded by Rajat Kumar Behera on 25 October 2018.

The user has requested enhancement of the downloaded file.

Big Data Ecosystem – Review on Architectural Evolution

Kamakhya Narain Singh[1,1], Rajat Kumar Behera[1], Jibendu Kumar Mantri[2]

[1]Kalinga Institute of Industrial Technology, Deemed to be University, Bhubaneswar
[2]Department of Computer Application, North Odisha University, Baripada
kamakhya.vphcu@gmail.com[1,1],rajat_behera@yahoo.com[1], jkmantri@gmail.com[2]

Abstract. Big Data is the collection of large datasets of different varieties, generated at an alarming speed with noises and abnormality. It is primarily popular for its five V's namely volume, variety, velocity, value and veracity. In an extremely disruptive world of open source systems that are playing a dominant role, big data can be referred to as the technology that can address the storage, processing and visualization of such data which is too diverse and fast-changing. It has led to a complex ecosystem of new frameworks, tools and libraries that are being released almost every day, which creates confusion as the technologists tussle with a swamp. This survey paper discusses the big data architecture and its subtleties that might help in applying the appropriate technology as a use case.

Keywords: Big Data, Big Data Ecosystem Architecture, Big Data Processing and Big Data Storage

1 Introduction

The concept big data has evolved due to outburst of data from various sources like data centers, cloud, internet, Internet of things (IoT), mobile, sensors and other spheres [1]. Data has entered into every industry, all business operations and currently thought about a major factor in production [2]. Big data is broad and encompasses new technology developments and trends [3]. It represents the enormous volume of structured, semi-structured and unstructured data and usually in terms of peta bytes (PB) or exa bytes (EB). The data are collected at an unparalleled scale and creates difficulty in making intelligent decisions. For instance, in the process of data acquirement, when the sourced data require decisions on cleanness i.e. what data to discard and what data to keep, remains a challenging task and how to store the reliable data with the right metadata becomes a major point of concern. Though the decisions can be based on the data itself, but greatly data are still not in a structured format. Blog content and Tweeter, Instagram feeds are imperceptibly structured pieces of text while machine generated data, such as satellite images, photographs, and videos are structured well for storage and visualization but not for semantic content and search. Transforming such content into a structured format for data analysis tends to be a problem. Undoubtedly, big data has the potential to help the industries in improving their operations and to make intelligent decisions.

1.1 The Five V's of Big Data

Every day, 2500 petabytes (PB) of data is created from digital pictures, videos, transaction records, posts from social media websites, intelligent sensors etc. [4]. Thus, big data describe massive and complex data sets which are unfeasible to manage with traditional software tools, statistical analysis and machine learning algorithms. Big data can be therefore characterized by the creation of data and its storage, analysis and retrieval as defined by 5 V [5].

1. Volume: It denotes to the enormous quantity generated in no-time and determines the value and potential of it under consideration and requires special computational platforms in order to analyze it.
2. Velocity: It refers to the speed at which data is created and processed to meet the challenges and demands that lie in the path of development and growth.
3. Variety:It can be defined as the type of the content of data analysis. Big data is not just the acquisition of strings, dates and numbers. It is also the data collected from various sources like sensors, audio, video, structured, semi-structured and unstructured texts.
4. Veracity: It is added by some organizations which focus on the quality of the variability in the captured data. It refers to the trust-worthiness of the data and the reputation of the data sources.
5. Value: It is referring to the significance of the data being collated for analysis. The proposition of the value is easy to access and produces various quality analytics like descriptive, diagnosis, prescriptive to produce insightful action in time-bound manner.

Additionally, two more V's which represent visualization and variability (i.e. constantly changing data) is commonly used to make it to 7V's but it fails to address additional requirements such as usability and privacy which are equally important.

1.2 Big Data Ecosystem Characteristics

A Big Data ecosystem must perform vigorous and be resource-efficient. Following are the desired characteristics of the big data ecosystem.

- Robustness and fault tolerance: Robustness is the ability of the system cope with the error and during the execution and also to cope with erroneous input. Systems need to work correctly and efficiently despite the machine failures. Fault tolerance is the ability of the system to continue operating correctly in the event of the failure of some of its components. The system must be robust enough to handle machine failures and human errors. The systems must be human-fault tolerance [6].
- Low latency reads and updates: It is the measurement of delay time or waiting time experienced by a system. As far as possible, the Big Data system has to deliver low read time and low update time [7].
- Scalability: It is the ability of a system to manage a growing amount of work or its potential to be enlarged to accommodate the growth. The Big Data

system has to promise for the highly scalable i.e. in the event of increasing data and load, computing resources should be plugged-in easily.

- Generalization: The Big Data System to support a wide spectrum of applications with the operational functions of all dataset [6].
- Extensibility: When needed, the Big Data System provision to add functionalities with minimized cost.
- Ad hoc queries: Big Data System should facilitate for ad hoc queries. As the need arises, the ad hoc queries can be created to obtain required information.
- Minimal Maintenance: Maintenance is defined as the work required in keeping the system runs smoothly. Big Data System with modest complexity should be prioritized [6] i.e. the maintenance of the system should be kept as minimal as possible.
- Debuggability: Debuggability is defined as the capability of being easily debugged. When required, a Big Data ecosystem must present the necessary granular information to debug [6] and also facilitate for the required extent to which something can be debugged.

2 Big Data Ecosystem Architecture

The architecture presented below is representing the evolution of big data architecture.

1. Lambda (λ) architecture: In the earlier days, big data systems were constructed to handle 3 V's of big data, namely Volume, Velocity and Variety to discover insights and make timely better business decisions. Nathan Marz coined Lambda Architecture (LA) to describe fault-tolerant (both against hardware failures and human mistakes), scalable and generic data processing architecture. LA aims to satisfy the needs for a strong, robust and healthy system by serving a wide range of workloads and use cases with low-latency reads and updates. LA also aims that resulting system should be linearly scalable, and should scale out rather than up [8]. The architecture uses a combination of batch and real time processing paradigm in parallel and runs on a real-time computational system. λ has three layers namely:

   Batch Layer: The batch layer is aimed to serve two fold purpose. The first purpose is to store the constantly growing immutable dataset into data sink and the second is to pre-compute batch views from the in-housed dataset. Computing the views is an ongoing operation, i.e. when new data arrives, it will be combined into the earlier existing views. These views may be computed from the entire dataset and therefore this layer is not expected to update the views frequently. Depending on the size of the in-housed dataset and cluster configuration, pre-computation could take longer time [19]. Batch layer produces a set of flat files containing the pre-computed views.

   Speed Layer: Whilst the batch layer continuously re-compute the batch views, the speed layer uses an incremental approach whereby the real time views are

incremented as and when new data is sourced [19] i.e. it manages only the recent data and computes the real-time views.

Serving Layer: Performs indexing and expose views for querying.

Incoming data are transmitted to both batch and speed layers for processing. The batch layer manages the immutable append-only raw data and then pre-computes the batch views. The speed layer deals with recent data and computes the real-time views. At the other end, queries are answered by assimilation of both batch and real-time views. Both layers execute the same processing logic and output results in a service layer. Batch views are batch write and random read wherein Real-time views are random write and random read [8]. Queries from back-end systems are executed based on the data in the service layer, reconciling the results produced by the batch and real-time views. The three layer architecture is outlined in Figure 1:

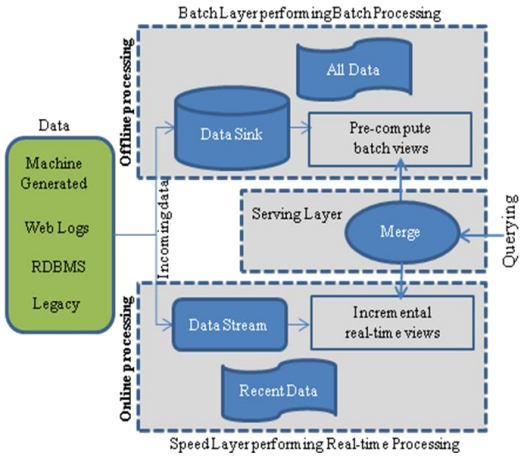


Fig. 1. Three layers of λ architecture, namely Batch, Speed and Serving Layer

Open-source technology stacks for λ architecture are presented in Table 1.

Table 1. λ Architecture Open Source Technology Stack

Area	Technology Stack
Data Ingestion	Apache Kafka, Apache Flume and Apache Samza
Batch Layer	Apache Hadoop, Apache MapReduce, Apache Spark and Apache Pig
Batch Views	Apache HBase, ElephantDB, and Apache Impala
Speed Layer	Apache Storm, Apache Spark Streaming
Real-time View	Apache Cassandra, Apache HBase
Manual Merge	Apache Impala
Query	Apache Hive, Apache Pig, Apache Spark SQL and Apache Impala

Error rectification of λ architecture is performed by allowing the views to be re-computed [9]. If error rectification is time consuming, the solution is to revert to the non-corrupted previous version of the data. This leads to a human fault tolerant system where toxic data can be completely removed and re-computation can be done

easily. The disadvantage of λ architecture is its complexity and its limiting influence. The batch and streaming processing pipeline require a different codebase that must be properly versioned and kept in sync so that processed data produces the same result from both paths [10]. Keeping in sync two complex distributed processing pipelines is quite maintenance and implementation. So a simpler that would bring the same benefits and handles the problem. So the essence is to develop λ architectures using a "unified" framework which makes the same codebase available to both the speed and batch layer and combines the results of both layers transparently, which leads to the development of unified λ architecture.

2. Unified λ architecture: It combines both batch and real-time pipeline, which runs concurrently and the results merged automatically [12]. From processing paradigms, the architecture integrates batch and real-time processing pipeline by offering a single API. The architecture is outlined in Figure 2.

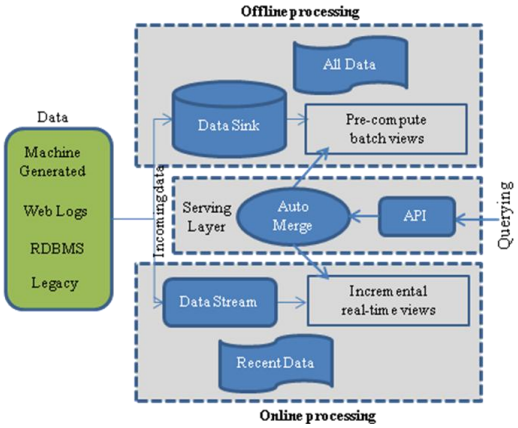


Fig. 2. Unified λ Architecture

With a unified framework, there would be only one codebase to maintain. Open-source technology stacks for Unified λ architecture is the replica of λ architecture except the "Auto Merge" area. Spring "XD" and Summingbird are the commonly used open source technology tool.

3. Kappa architecture: It is the simplification of λ architecture [13]. Kappa architecture is similar to λ architecture with the removal of batch processing paradigm. In 2014 summer, Jay Kreps posted an article addressing pitfalls associated with λ architecture [14]. Kappa architecture is avoiding maintaining two separate codebases of batch layer and speed layer. It is handling real-time data processing and continuous data re-processing using a single stream processing computation model. Hence, it consists of two layers, namely speed and serving layer. The speed processing layer runs the stream/online processing jobs. Usually, a single online processing job is run to enable real-time data processing. Data re-processing is done when some code of the online processing job needs to be tweaked. This is accomplished by running another changed online

processing job and replaying all previous housed data. Finally, the serving layer is used for querying [14]. The architecture is outlined in Figure 3.

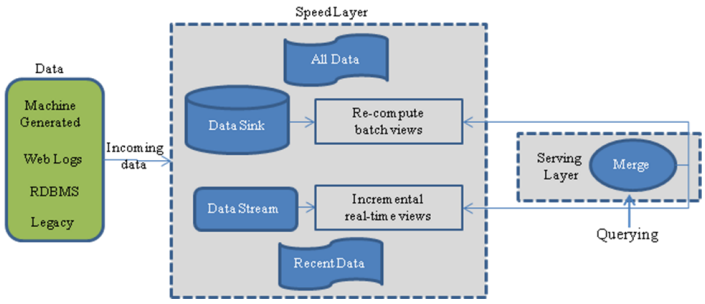


Fig. 3. Kappa Architecture

Open-source technology stacks for Kappa architecture are presented in Table 2.

Table 2. Kappa Architecture Open Source Technology Stack

Area	Technology Stack
Data Ingestion	Similar to λ Architecture Open Source Technology Stack
Batch Views	Apache HBase, ElephantDB
Speed Layer	Apache Storm, Apache Spark Streaming
Real-time View	Apache Cassandra
Queries	Apache Hive, Apache Pig and Apache Spark SQL

4. Microservices architecture: It divides big data system into many undersized services called microservices that can run independently. This allows every service to run its own process and communicate in a self-ruling way without having to depend on other services or the application as a whole [15]. Martin Fowler defines that the services must adhere to the common architectural principles, including single responsibility, separation of concerns, don't repeat yourself (DRY), composability, encapsulation, loose coupling, use of consistent and standardized interfaces etc [16]. The architecture is outlined in Figure 4.

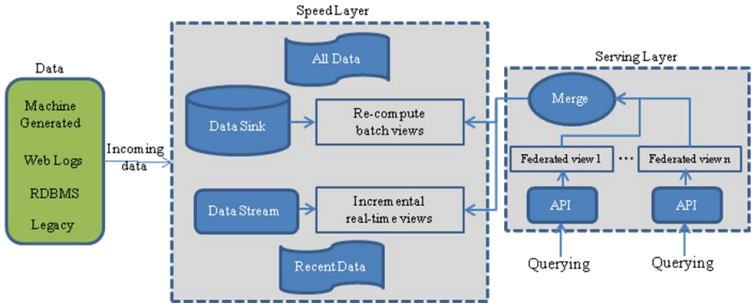


Fig. 4. Microservices Architecture

Open-source technology stacks for Microservice architecture are identical to the λ Architecture except for Real-time View and Federated View. Apache Cassandra is the

commonly used open source technology tool for real-time view and Apache Phoenix is the commonly used open source technology tool for federated view.

5. Mu architecture: It ingests data into both batch and streaming process, but not for reliability viewpoint, because some work is better done in batches and some are in streaming paradigm [17]. The architecture is outlined in Figure 5.

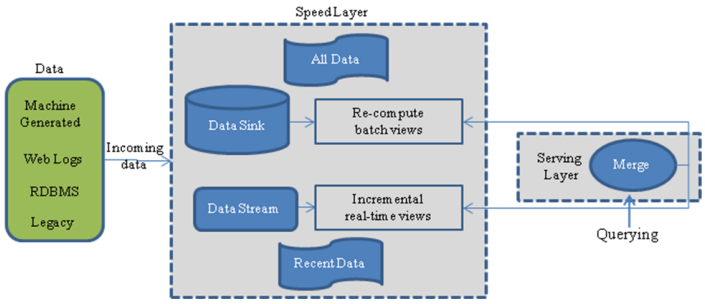


Fig. 5. Mu Architecture

Open-source technology stacks for Mu architecture remains same as Kappa architecture.

6. Zeta architecture: It is built on pluggable components and all together, it produces a holistic architecture [18]. Zeta is characterized by 7 components, namely:

    1. Distributed File System (DFS): It's the common data location for all needs and is reliable and scalable.
    2. Real-time Data Storage: it's based on real-time distributed technologies, especially NoSQL solutions and is meant for delivering user supplied responses promptly and quickly.
    3. Enterprise Applications (EA): EA focuses to comprehend all business goals of the system. The examples of this layer are web servers or business applications.
    4. Solution Architecture (SA): SA spotlight is on a specific business problem. Unlike EA, it concerns a more specific problem. Different solutions can be combined to construct the solution for the more global problem.
    5. Pluggable Compute Model (PCM): it implements all analytic computations and are pluggable in nature as it has to cater to different needs.
    6. Dynamic Global Resource Management (DGRM) - It allows dynamic allocation of resources that enables business to easily accommodate for priority tasks.
    7. Deployment/Container Management System: this guarantee a single, standardized method of deployment and implies that deployed resources don't concern about any environment changing, i.e. deployment in the local environment is identical with prod environment.

The architecture is outlined in Figure 6.

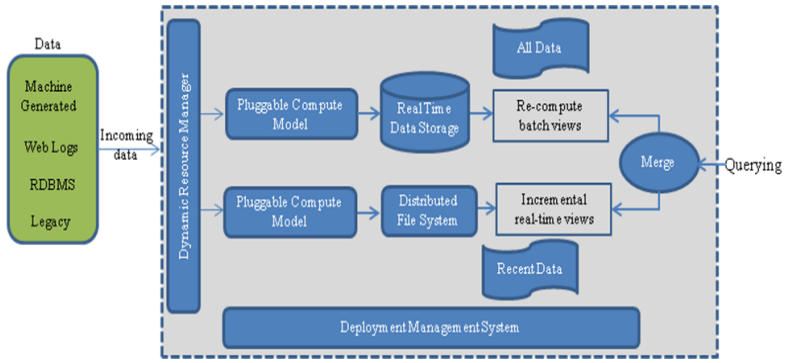


Fig. 6. Zeta Architecture

Open-source technology stacks for Zeta architecture are presented in Table 5.

Table 3. Zeta Architecture Open Source Technology Stack

Area	Technology Stack
Data Ingestion	Similar to λ Architecture Open Source Technology Stack
DGRM	Apache Mesos and Apache YARN
PCM	Apache Spark and Apache Drill
Real Time Data Storage	Apache HBase and Couchbase
DFS	HDFS
Batch View	Similar to Kappa Architecture Open Source Technology Stack
Real-time View	Apache Cassandra
Queries	Similar to Kappa Architecture Open Source Technology Stack

7. IOT architecture (iot-a): The Internet of Things (IoT) is an ecosystem of connected devices and possibly human, accessible through internet. The IP address is assigned to the devices and it collates, transfer data over the internet without manual and human intervention. Examples of such devices are RFID sensors and sophisticated devices like smartphones. Data from such IoT devices transmitted to Big Data Ecosystem to produce a continuous stream of informtion. In simplicity, IoT devices generates the data and is delivered to Big Data Ecosystem to generate insight in time bound manner [20]. The Big Data Ecosystem uses scaling-out approach on commodity hardware to overcome the challenges posed by such devices. iot-a is composed of three primary building blocks namely:
    - Ad hoc queries: Message Queue/Stream Processing (MQ/SP): It receives data from upstream system and depending on the business mandate, performs buffering, filtering and complex online operations
    - Database: It receives data from MQ/SP and provides structured and low-latency access to the data points. Typically, the database is a NoSQL solution with auto-shading and horizontal scale-out properties. The database output is of interactive nature, with an interface provided either through a store-specific API or through the standard interface SQL [20].

- Distributed File System (DFS): In general, it receives data from either the DB and performs batch jobs over the entire dataset. If required, it can also receive data directly from MQ/SP block. This might include merging data from IoT devices with other data sources [20].

iot-a architecture is outlined in Figure 7.

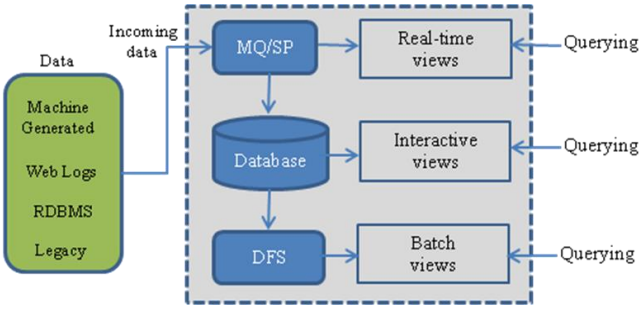


Fig. 7. iot-a Architecture

Open-source technology stacks for iot-a architecture are presented in Table 6.

Table 4. iot-a Architecture Open Source Technology Stack

Area	Technology Stack
Data Ingestion	Similar to λ Architecture Open Source Technology Stack
Real-time views	Apache Cassandra
Interactive Processing	Apache Spark
Interactive views	Apache Drill
Batch Processing	Apache Mahout
Batch views	Apache Hive
Queries	Apache Hive, Apache Cassandra and Apache Drill

3 Discussion

This paper briefly reviews big data ecosystem architecture to the best of the knowledge, for discussions and usages in research, academia and industry. The information presented discusses some research papers in the literature and a bunch of systems, but when it comes to the discussion of a small fraction of the existing big data technology and architecture, there are many different attributes that carry equal importance, weight and a rationale for comparison.

4 Conclusion

A theoretical study or a survey of various tools, libraries, languages, file systems, resource managers, schedulers, search engines, SQL and NoSQL frameworks, operational and monitoring frameworks had been highlighted in order to provide the researcher with the information for understanding Big Data Ecosystem which are on a

rapid growth raising concerns in terms of business intelligence and scalable management that includes fault tolerance and optimal performance. In this paper, big data architecture has been discussed though not in an elaborate manner, but hope this paper will serve as a helpful introduction to readers interested in big data technologies.

References

1. Salisu Musa Borodo, Siti Mariyam Shamsuddin, Shafaatunnur Hasan. "Big Data Platforms and Techniques", Vol. 17, No. 1, January 2016, pp. 191 ~ 200.
2. James M, Michael C, Brad B, Jacques B, Richard D, Charles R. Big data: The next frontier for innovation, competition, and productivity. McKinsey Glob Inst. 2011.
3. 10 emerging technologies for Big Data – TechRepublic, 2012, http://www.techrepublic.com/blog/big-data-analytics/10-emerging-technologies-for-big-data/
4. Every Day Big Data Statistics – 2.5 Quintillion Bytes of Data Created Daily, 2015, http://www.vcloudnews.com/every-day-big-data-statistics-2-5-quintillion-bytes-of-data-created-daily/
5. Big Data: The 5 Vs Everyone Must Know, 2014, https://www.linkedin.com/pulse/20140306073407-64875646-big-data-the-5-vs-everyone-must-know
6. Notes from Marz' Big Data – principles and best practices of scalable real-time data systems – chapter 1, 2017, https://markobigdata.com/2017/01/08/notes-from-marz-big-data-principles-and-best-practices-of-scalable-real-time-data-systems-chapter-1/
7. The Secrets of Building Realtime Big Data Systems, 2011, https://www.slideshare.net/nathanmarz/the-secrets-of-building-realtime-big-data-systems/15-2_Low_latency_reads_and
8. lambda architecture, 2017, http://lambda-architecture.net/
9. Big Data Using Lambda Architecture, 2015, http://www.talentica.com/pdf/Big-Data-Using-Lambda-Architecture.pdf
10. wikipedia lambda architecture, https://en.wikipedia.org/wiki/Lambda_architecture
11. Kreps, Jay. "Questioning the Lambda Architecture", 2014, radar.oreilly.com
12. Lambda Architecture for Big Data by Tony Siciliani, 2015, https://dzone.com/articles/lambda-architecture-big-data
13. Kappa architecture, http://milinda.pathirage.org/kappa-architecture.com/
14. Data processing architectures – Lambda and Kappa, 2015, https://www.ericsson.com/research-blog/data-processing-architectures-lambda-and-kappa/
15. Microservices Architecture: An Introduction to Microservices, 2017, http://www.bmc.com/blogs/microservices-architecture- introduction-microservices/
16. Data Integration Design Patterns With Microservices by Mike Davison, 2016, https://blogs.technet.microsoft.com/cansql/2016/12/05/data-integration-design-patterns-with-microservices/
17. Real Time Big Data #TD3PI, 2015, http://jtonedm.com/2015/06/04/real-time-big-data-td3pi/
18. Zeta architecture, 2017, http://www.waitingforcode.com/general-big-data/zeta-architecture/read
19. The Lambda architecture: principles for architecting realtime Big Data systems, http://jameskinley.tumblr.com/post/37398560534/the-lambda-architecture-principles-for
20. iot-a : the internet of things architecture, http://iot-a.info

View publication stats